プログラミング３ 講義資料

第11回 チーム開発③：詳細設計

- 処理フローの記述
- モジュール分割と I/O
- Issue → PR の運用ルール

目次

- 詳細設計
- Issue/PR の本格運用

詳細設計

今回の目的と到達目標

今回の目的

- 処理フローを書ける
- モジュール境界を切れる
- PR 運用を開始する

到達目標

- [3] チーム開発での協調作業

詳細設計とは

- 基本設計の中身を **実装に落とせる粒度まで** 具体化する
- 各機能について「どの関数に / どの順で / どのデータを渡すか」を決める

設計段階	粒度
要件定義	ユーザーに何を提供するか
基本設計	画面・DB・構成の全体像
詳細設計	関数・モジュール単位の処理

処理フローの記述

- **1機能につき1処理フロー** を書く
- 書き方: 番号付き手順 + 分岐条件

例: 「投稿作成」の処理フロー

1. フォームから title / body を受け取る
2. 入力バリデーション（空チェック・長さ）
3. NG → エラーメッセージを返す
4. OK → DB に `posts` レコードを作成
5. 作成結果を詳細画面にリダイレクト

モジュール分割の観点

- **単一責任**: 1モジュール = 1つの関心
- **疎結合**: モジュール間の依存を最小にする
- **I/O を明示**: 入力 → 処理 → 出力 を関数シグネチャで表す

モジュール	入力	出力
auth	email, password	user_id or null
post_repo	post_data	post_id
renderer	post	HTML文字列

Issue/PR の本格運用

Issue → 実装 → PR の流れ

1. Issue を選んでアサイン
2. feature ブランチを作成（`feature/<issue番号>-<短い説明>`）
3. 作業 → コミット → push
4. GitHub 上で PR 作成（本文に `closes #<issue番号>`）
5. レビュー依頼 → 修正 → Approve → merge
6. feature ブランチ削除、Issue は自動クローズ

> 1 Issue = 1 PR を守ると履歴が追いやすい

PR を小さく保つ

- 目安: 数百行以内、1テーマのみ
- 大きすぎる PR は:
  - レビュワーが読み切れない
  - コンフリクトが起きやすい
  - バグ混入時の特定が困難
- 大きな機能は **複数 PR に分割** する
- 「下準備」と「本機能」を別 PR にするのが定石

レビューを早く回す工夫

Tips

- Draft PR で作業中でも進捗共有
- セルフレビュー後にレビュー依頼を出す
- 依頼には「特に見てほしい箇所」を書く
- 24時間以内にレスポンスする運用を決める
- 小さな修正は即マージ、議論が長引くものは別Issueに切り出す

今回のまとめ

- 詳細設計で処理フローとモジュール境界を具体化
- 1 Issue = 1 PR、feature ブランチ → レビュー → merge の運用
- PR は小さく、レビューを早く回す

今回カバーした MCC 項目

- V-D-4 コンピュータシステム

次回予告

- 第12回: チーム開発④ 実装・PRレビュー・AI活用記録